Rheinisch-Westfälische Technische Hochschule Aachen

Bachelor Thesis

A Very Nice Bachelor Topic

by

Jan-Christoph Kassing

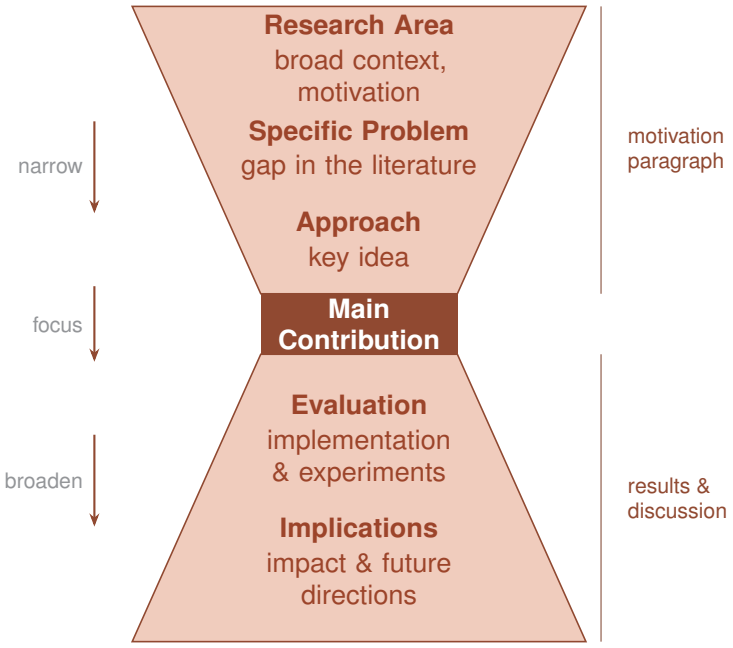
Lehr- und Forschungsgebiet Informatik 2
Programming Languages and Verification

First Examiner:
Prof. Dr. Examiner 1

Second Examiner:
Prof. Dr. Examiner 1

April 27, 2026

Abstract



Hourglass structure of a good abstract. Start broad (research area and problem), funnel down to the specific contribution, then widen again to state results and implications.

Write a self-contained summary of roughly 150–250 words following the hourglass structure above. Cover the following five points in order:

1. **Research area.** Name the field (e.g., automated termination analysis, probabilistic verification, complexity analysis of rewrite systems) and state in one sentence what the field studies and why it matters.

2. **Specific problem.** State precisely which problem this thesis addresses and why it is challenging or not yet fully solved by existing approaches.

3. **Approach.** Describe the key idea of your contribution in one or two sentences. Avoid mathematical notation and abbreviations, and use informal language.

4. **Theoretical results.** Name the main property proved (soundness, completeness, an equivalence result, or a complexity bound) and state it informally.

5. **Implementation and evaluation (if applicable).** Name the tool and the benchmark set (e.g., the Termination Problems Data Base) and give the headline result: how many more examples are solved compared to the baseline and what is the runtime overhead.

6. **Implications.** Explain how your results solved the issue or fille the gap you pointed out beforehand.

Acknowledgements

I want to express my most profound appreciation to my supervisor!

Maybe I want to thank someone else as well...

Contents

Abstract **iii**

Acknowledgements **v**

Contents **vi**

1 Introduction **1**

2 Preliminaries **3**
2.1 Basic Definitions 3
2.2 Known Results from the Literature 4

3 [*Title of Your Main Contribution*] **6**
3.1 [*Central Definitions / The New Framework*] 6
3.2 [*Main Results*] 6

4 Evaluation **8**
4.1 Implementation 8
4.2 Experimental Setup 8
4.3 Results 9
4.4 Discussion 9

5 Conclusion **10**

Bibliography **11**

CHAPTER 1

Introduction

The introduction motivates the topic and provides an overview of the thesis. It can be seen as a longer version of the abstract that follows the hourglass structure again, but with emphasis on the top part of the hourglass. However, you can now use citations and explain everything in more detail. It should be accessible to any computer scientist without prior knowledge of the specific area.

The background and motivation part should cover the following points:

1. Name the research area and describe informally what objects are studied (e.g., terms, integer programs, Java programs, automata) and what property is of interest (e.g., termination, complexity, safety).
2. Explain why the property matters.
3. Give a small, self-contained example that illustrates the problem informally.
4. Explain why answering the question automatically is non-trivial and what makes the problem hard in general.

The open-problem part should contain:

1. A summary of what is already known in the area (prior results, existing automated techniques, tool support).
2. A precise identification of the gap: e.g., no automated procedure exists, existing techniques are unsound or incomplete for a relevant subclass, or no efficient algorithm is known.
3. A clear statement of what this thesis aims to achieve.

Related Work

Discuss the most relevant prior work. For each cited work, address the following:

1. What does the work do? Describe the approach in one to three sentences and do not just list references.
2. How does it relate to this thesis? Does it solve a special case, use a different technique, or target a different formalism?
3. What limitation motivates your own contribution?

Contribution

List one item per concrete contribution. Each item should:

1. Identify what is new (a technique, a criterion, a framework, an implementation, or an experimental evaluation).
2. State the property proved, if applicable (soundness, completeness, both, or a complexity bound).
3. Reference the theorem, or section in which the contribution appears.

Structure

End the introduction by outlining the remaining chapters. Give at least one sentence per chapter describing its content. Do not simply write "Chapter X presents the main results."

Example: "The remainder of this thesis is organized as follows. Chapter 2 introduces the formal background required throughout this thesis, including ... Then, Chapter 3 presents ... Chapter 4 describes the implementation and the experimental evaluation. Finally, Chapter 5 summarizes the results and discusses directions for future work."

CHAPTER 2

Preliminaries

The opening paragraph of a chapter should briefly outline the chapter structure by naming each section and stating what it covers.

In the preliminaries you can name the formalism studied (e.g., term rewriting systems, integer transition systems, probabilistic programs) and cite a standard reference for further reading, e.g., [1]. Do not add a citation to every definition.

2.1 Basic Definitions

Introduce the formal objects of the theory bottom-up: start with the most elementary notion (e.g., the syntactic objects) and build towards the property of interest (e.g., termination, complexity).

Each definition in this section should be accompanied by:

1. A sentence stating what the definition formalizes and why it is needed.
2. The formal statement in a Definition box.
3. A brief explanation of each component of the definition.
4. At least one example.

Cover the following in order:

1. **Syntactic Objects.** Define the elementary building blocks of the formalism (e.g., the signature and the set of terms, the program syntax, the set of states and transitions).
2. **Evaluation Relation.** Define how one object evaluates to another. If several strategies exist, introduce them here and state which one is used in the rest of the thesis.
3. **Derived Notions.** Define derived concepts such as normal forms, reachability, derivation lengths, etc.
4. **The Property of Interest.** Give the formal definition of the property the thesis analyzes (e.g., termination, innermost complexity, almost-sure termination).

In the begging, we start by defining [Basic Notion] that is used throughout the remainder of this thesis.

Definition 2.1.1: [Name of First Basic Notion]

A [Thing] is [Basic Notion] if it satisfies [Property 1] and [Property 2].

The reason for requiring [Property 1] is [...]. [Property 2] is generally assumed in the literature.

Example 2.1.2.

Example 2.1.3.

2.2 Known Results from the Literature

Present the lemmas and theorems from the literature that are used in Chapter 3. You do not have to prove these results unless the proof is insightful for the rest of the thesis. Always cite the original source. Each result in this section should be presented as follows:

1. A sentence explaining why the result is important and how it will be used in the thesis.
2. The formal statement in a Lemma or Theorem box, with a citation in the title.
3. A proof indication: either a short proof, "Proof: see [**?**]", or a forward reference to the appendix.
4. (Optional) A brief remark on the scope or limitations of the result if that is relevant for understanding its use in Chapter 3.

This section collects known results from the literature that are used in the rest of the thesis. Organize them as follows:

1. **Auxiliary lemmas.** State small technical results about the basic objects from Section 2.1 (e.g., properties of substitution, closure conditions, or monotonicity). These are typically needed in the proofs of Chapter 3.
2. **Characterization theorem(s).** State the main theoretical results from the literature about the property of interest. These serve as a reference point for understanding what is already known.
3. **Existing automated technique(s).** If the thesis extends an existing automated method (e.g., dependency pairs, polynomial interpretations, ranking functions), state its correctness theorem here.

Lemma 2.2.1: [Name of Auxiliary Lemma], [2]

A Lemma is a small result or auxiliary result, which is only interesting because we can use it to prove the main result.

Proof. The complete proof can be found in [2]. □

Theorem 2.2.2: [Name of Key Theorem], [2]

A Theorem is one of the main results. Everything else is defined and proven such that this results can be proven.

Proof. The complete proof can be found in [2]. □

Example 2.2.3. Apply the theorem above to a concrete example. Demonstrate both a case where the result applies successfully and, if possible, a case where it fails due to the restriction identified above. The failing case motivates the extension in Chapter 3.

Corollary 2.2.4: [Name of a Direct Consequence], [2]

A Corollary is a (easy) consequence of some other (main) result. Still, it has to be proven, even if the proof is rather simple!

Proof. The complete proof can be found in [2]. □

Example 2.2.5.

You can use colors to highlight differences in boxes or even difficult parts in proofs. However, do not make the colors too distracting and everything has to be readable in black and white as well.

CHAPTER 3

[*Title of Your Main Contribution*]

This chapter contains the main theoretical (and possibly technical) contribution of the thesis. It should present a coherent body of new results, organized into sections.

For a theory-focused thesis: introduce new notions, state lemmas/theorems, provide proofs, and illustrate results with examples. For an implementation-focused thesis: additionally describe the algorithm, key design decisions, and data structures.

The opening paragraph of this chapter should cover the following points:

1. State what new objects, criteria, or techniques this chapter introduces.
2. Introduce a running example that will be used throughout to illustrate each new definition and result.
3. Outline the chapter structure by referencing the main sections.

3.1 [*Central Definitions / The New Framework*]

For self-introduced notions it is even more important to explain everything and used multiple examples to illustrate everything. Use one running example that can be used through the whole section and maybe some additional small examples to highlight special cases.

Definition 3.1.1: [*Name of Your First Definition*]

Example 3.1.2.

3.2 [*Main Results*]

Each result in this section should be presented as follows:

1. State and prove all auxiliary lemmas before the main theorem.
2. State the main theorem. Typical statements are:
   - **Soundness:** If the technique returns YES, then the property holds.
   - **Completeness:** If the property holds, the technique returns YES.
   - **Equivalence:** The new criterion holds if and only if an existing criterion holds.

3. Provide a complete proof, structured into numbered cases for long arguments. Reference definitions and earlier results by label.
4. Apply the theorem to the running example from Example 3.1.2, walking through every step explicitly.

Lemma 3.2.1: [*Name of Auxiliary Lemma*]

Proof. □

Theorem 3.2.2: [*Name of Main Theorem, e.g., Soundness*]

Proof. □

Example 3.2.3. Apply Theorem 3.2.2 to the running example from Example 3.1.2. Walk through every step of the argument explicitly.

CHAPTER 4

Evaluation

This chapter demonstrates that the approach from Chapter 3 works in practice.

A good evaluation chapter:

- Describes clearly what was implemented and where.
- Explains the experimental setup with enough detail to be reproducible.
- Presents results honestly, including cases where the technique does not help.
- Interprets the numbers and connects them back to the theory.

4.1 Implementation

The implementation section should cover:

1. The name of the tool extended and a brief description of its overall architecture (e.g., the pipeline from input parsing to proof output). and any external solvers used.
2. Which module(s) you added or changed and how they fit into the existing architecture.

4.2 Experimental Setup

The experimental setup should specify:

1. **Benchmark set.** Which set was used (e.g., TPDB, TermComp, SV-COMP, or a custom set)? How many examples total? Are there relevant subsets by category, strategy, or problem type?
2. **Configurations compared.** At minimum include the unmodified baseline and the tool with your extension. List any competing tools with a brief description.
3. **Hardware and time limits.** CPU model, amount of RAM, operating system, and per-example wall-clock timeout.

4.3 Results

The results section should:

1. Present results in a table (see Table 4.1).
2. Summarize the headline finding directly below the table, citing concrete numbers (additional YES answers, percentage improvement, overhead in seconds).
3. Point out interesting subsets or individual examples where the technique is especially effective and where the technique is not effective.
4. Note any surprising results to be discussed in Section 4.4.

Table 4.1 summarizes the results of our experiments.

Table 4.1: Experimental results on [*benchmark set name*]. *YES/NO* denotes the number of examples for which [*the property / its negation*] was proved. *MAYBE* means the tool returned no answer. *TO* means the timeout was exceeded. *Time* is the total wall-clock time in seconds.

Configuration	**YES**	**NO**	**MAYBE**	**Avg. Time (s)**
Tool without extension	–	–	–	–
Tool with extension	–	–	–	–

4.4 Discussion

The discussion should address the following questions:

1. Do the results match theoretical expectations from Chapter 3? If not, explain why.
2. On which types of examples does the technique help most? Identify structural properties of the benchmark examples that explain the gain.
3. Are there cases where the technique does not help or is slower? Explain the reason and relate it to the theoretical properties.
4. What are the main limitations and how could they be overcome? (This can bridge to the future work discussion in Chapter 5.)

CHAPTER 5

Conclusion

The conclusion summarizes the thesis and discusses future work. Do not introduce any new material here. A reader who reads only the introduction and the conclusion should get a coherent picture of what was done and what remains open. It should follow the same structure as the introduction and the abstract, but with more emphasis on the bottom part of the hourglass.

The summary should address the following points:

1. Restate the problem studied and explain why it is relevant.
2. Describe the main contribution and the approach taken (reference Chapter 3).
3. Name the central theoretical result (reference Theorem 3.2.2) and state informally what it guarantees.
4. Summarize the experimental findings – number of additional examples solved, runtime overhead – if an evaluation was conducted (reference Chapter 4).

Each future work item should:

1. Identify a specific limitation of the current approach or a natural extension that was beyond the scope of this thesis.
2. Describe what would be needed to address it (new theoretical machinery, a different encoding, a richer formalism, etc.).
3. Explain what benefit this would bring (more examples solved, a stronger guarantee, applicability to a wider class of systems).

Bibliography

[1] Franz Baader and Tobias Nipkow. *Term rewriting and all that*. 1999.

[2] Jürgen Giesl, René Thiemann, Peter Schneider-Kamp, and Stephan Falke. Mechanizing and improving dependency pairs. *Journal of Automated Reasoning*, 37:155–203, 2006.